Class Weights for Imbalanced Data

2026-04-17

Introduction

Class weighting addresses imbalanced classification by scaling each observation's loss contribution inversely to its class frequency, giving the minority class more influence on the gradient or split decision. It achieves similar effects to SMOTE-style oversampling but without synthesising new data: simpler, deterministic, easier to reproduce, and natively supported by most classifiers (logistic regression, random forest, XGBoost, neural networks). For most imbalanced problems, class weights are the right starting point before reaching for resampling.

Prerequisites

A working understanding of classification loss functions, the accuracy paradox in imbalanced classification, and basic familiarity with resampling alternatives (oversampling, undersampling, SMOTE).

Theory

The weighted loss is

$$L = \sum_i w_{y_i} \cdot \ell\big(f(x_i), y_i\big),$$

where w_k is the weight for class k. Typical balanced weights use the inverse class frequency: $w_k = n/(Kn_k)$ where K is the number of classes and n_k is the count of class k. This makes the total weight per class equal, so each class contributes equally to the loss regardless of its sample size.

In tree-based methods, class weights enter the split-impurity computation; in gradient-boosted methods, they enter the gradient and Hessian; in neural networks, they multiply the per-example loss before summing.

Assumptions

The problem is genuinely a classification task with imbalanced classes; the imbalance reflects the population (not just an artefact of training-data collection that will not transfer to deployment).

R Implementation

```
library(ranger)

set.seed(2026)
n <- 500
df <- data.frame(
  x1 = rnorm(n), x2 = rnorm(n),
  y  = factor(sample(c("pos", "neg"), n, replace = TRUE,
                     prob = c(0.05, 0.95)))
)
```

```
idx <- sample(n, n * 0.7)
train <- df[idx, ]; test <- df[-idx, ]

rf_unw <- ranger(y ~ ., data = train, num.trees = 200,
                 classification = TRUE)
pred_unw <- predict(rf_unw, test)$predictions

# Inverse-frequency class weights
cw <- setNames(as.numeric(1 / prop.table(table(train$y))),
               names(table(train$y)))
rf_w <- ranger(y ~ ., data = train, num.trees = 200,
               classification = TRUE, class.weights = cw)
pred_w <- predict(rf_w, test)$predictions

table(truth = test$y, unweighted = pred_unw)
table(truth = test$y, weighted   = pred_w)
```

Output & Results

Two confusion matrices: the unweighted random forest predicts almost exclusively the majority class (the rational behaviour given 95:5 imbalance and accuracy-maximising training); the weighted RF recovers more minority-class examples at the cost of a few additional false positives.

Interpretation

A reporting sentence: "Inverse-frequency class weighting on the 95:5 imbalanced dataset improved minority-class recall from 0.12 to 0.56 with a modest decrease in majority precision (0.97 to 0.93); the weighted classifier is more useful for screening applications where missing minority cases is more costly than false positives." Always state the weighting scheme and the trade-off in terms of the application's cost structure.

Practical Tips

- Always try class weights before SMOTE-style oversampling; they are deterministic, faster, and produce reproducible results.
- Tune weights by cross-validation; inverse-frequency is a sensible default but not necessarily optimal for the cost structure of every application.
- All major classifiers support class weights natively: `glmnet(weights = ...)`, `ranger(class.weights = ...)`, `xgboost(scale_pos_weight = ...)`, `keras::compile(... loss_weights = ...)`.
- Weighting shifts the decision boundary; inspect calibration plots and ROC curves to check probabilistic quality after weighting.
- For severely imbalanced problems (below 1 %), combine class weights with threshold tuning on a validation set; the default 0.5 cutoff is rarely optimal under heavy imbalance.
- For problems where the cost of errors is asymmetric and known, build a cost-sensitive classifier directly (`mlr3::msr("classif.costs")`) rather than tuning weights heuristically.

R Packages Used

`ranger` for random forest with native class weights; `glmnet` for penalised logistic regression with `weights = ...`; `xgboost::xgboost(scale_pos_weight = ...)` for gradient-boosted trees; `themis` for SMOTE-style alternatives within tidymodels; `parsnip::set_args(class_weights = ...)` for class-weight support across multiple engines.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis